

Partitioning

- Specification -> potentially concurrent cooperating processes.
- Assign process to some resource
- Process description
 - Executable model
 - Document
 - Legacy design
- Resource
 - Processor
 - Silicon area
 - IP block
 - Communication channel
 - Storage resource

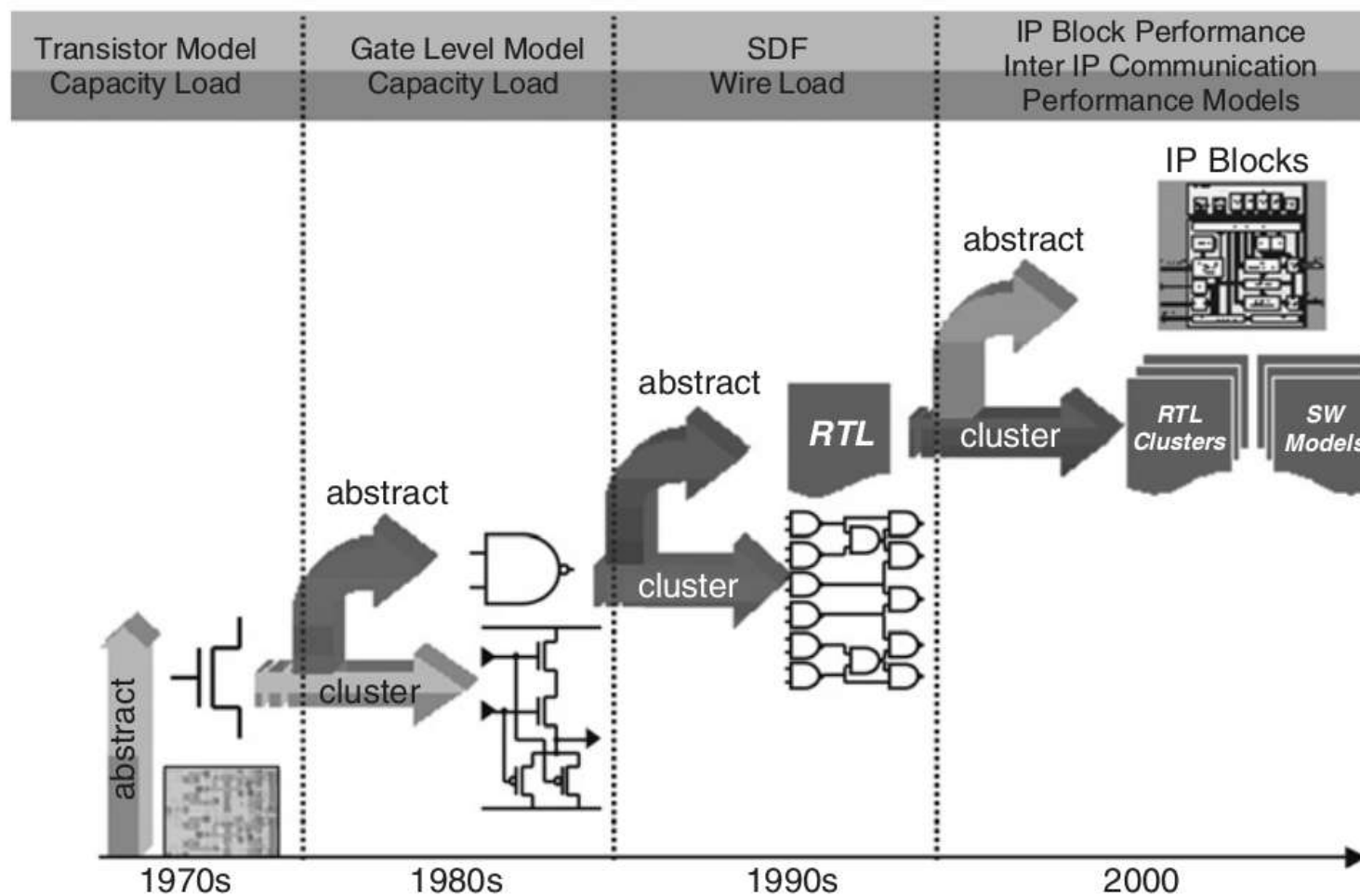
Divide and conquer paradigm

- Pro - Independent teams
- Pro - Interfaces introducing simplify the unit verification
- Con - Reduces optimization opportunities

Complexity Reduction

- Abstraction and clustering
- Abstraction - Ignore low-level details
- Clustering - Connect a set of objects to get a new object
- Mapping - Abstract object translation to cluster

Partitioning - Clustering



Partitioning - Platform

- The platform allows design developers to work at the higher level - applications
- Application maps to the platform objects
- No need for detailed implementation knowledge.
- Implementers share design and fabrication costs

Steps

- ① Functional decomposition
 - Monolithic specification -> functional components identification
 - Functionality - Application
- ② Architecture description
 - The target architecture definition
 - Platform
- ③ Assignment
 - Functional units -> Architectural units
 - Partitioning, Mapping
- ④ Detailed interface definition
 - Pre-partitioning analysis guided
 - Account for constraints
 - Automate if possible
 - Progressively more difficult as we move upward

Functional Decomposition I

- Effective FD -> Specification without implementation side effects
- Executable vs Natural Language
 - Unintended ambiguities
 - No implementation side effects
- Performance -> Parallelism
- Parallelism -> Better assignment
- Expose parallelism
 - Functional concurrent language (such as Simulink)
 - Automatically Extract Parallelism

Automatically Extract Parallelism

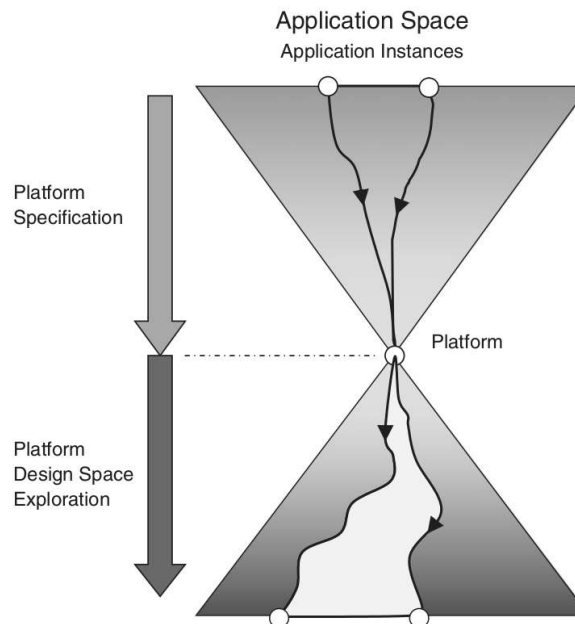
- Promising and difficult
- Pros: Legacy sequential descriptions
- The dependency analysis under many restrictions
- Hardware-dependent solutions

Architecture Description

- An architecture - component collection - an abstract schematic
- The components are both hardware and software:
 - CPU (Fixed - Configurable)
 - Hardware (Custom - Standard)
 - Buses
 - Memories
 - Operating systems (RTOS)
 - Software services
 - APIs
 - Protocol stacks (Network protocol)
 - Software object components
- Component type
 - Computation
 - Communication
 - Storage
- The platform is a semi-finished solution for describing architecture.

Platforms

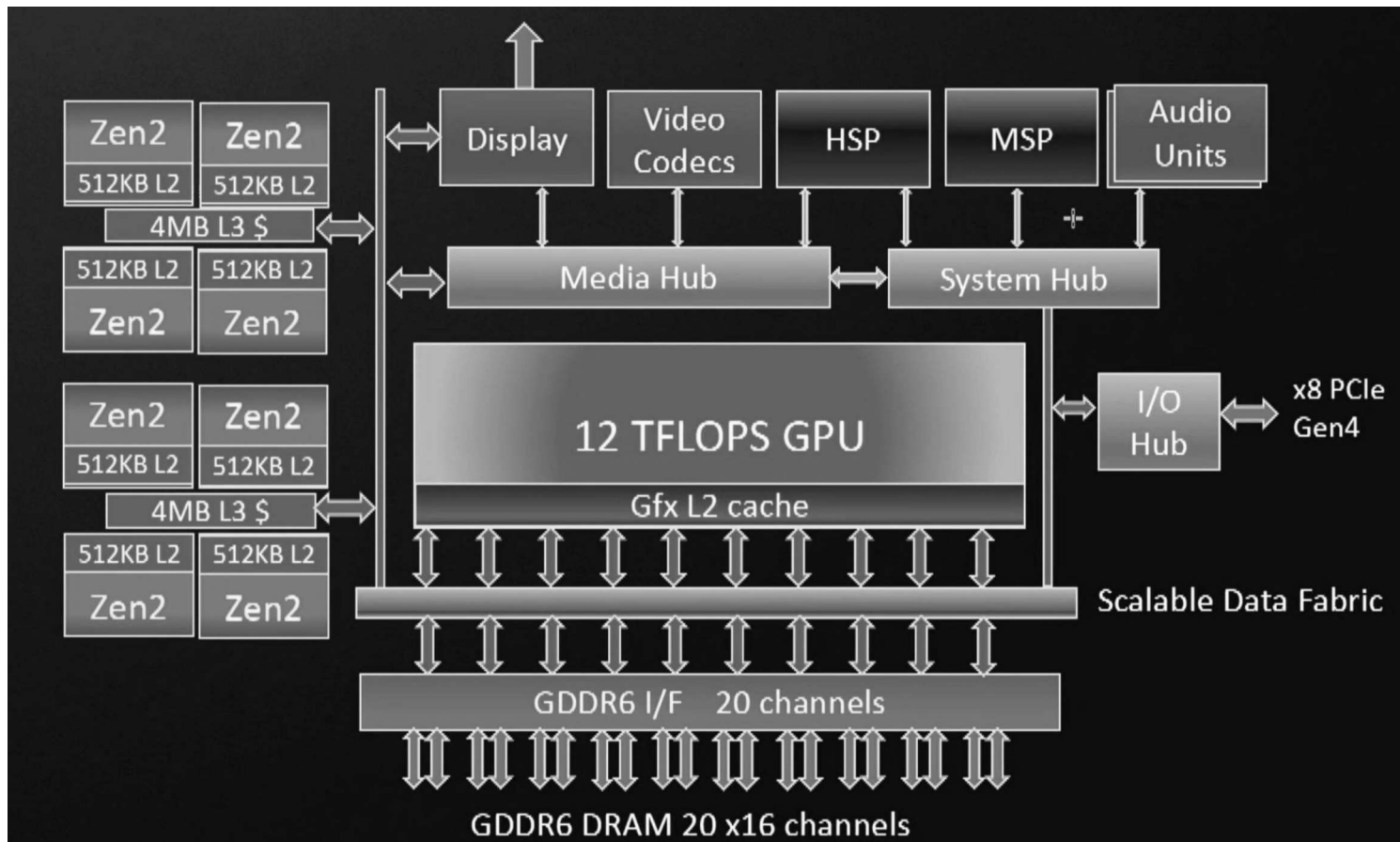
- Platforms-parameterized HW/SW topologies - configurable for a range of applications
- Time-to-market -> existing architectures
- Almost all ESL-based designs are iterations of existing platforms
- Several different designs share the base architecture
- Given an application domain - limited good solutions number
- Architecture that best captures these good solutions



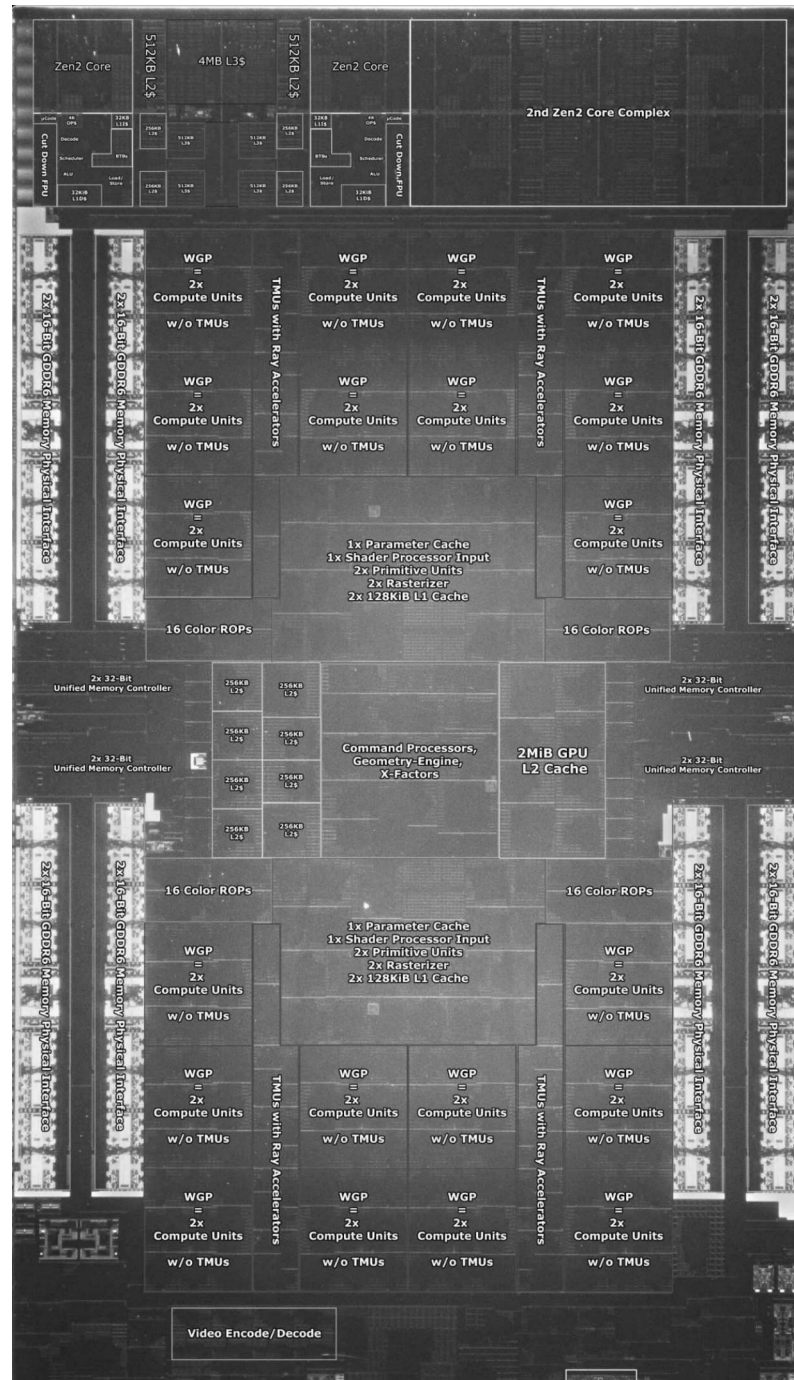
Platforms Example

- Complete SoC solutions
 - Qualcomm Snapdragon
 - Apple Silicon
 - MediaTek Dimensity
 - Samsung Exynos
- Semi-custom solutions
 - Xbox Series X
 - PS5

Xbox Series X



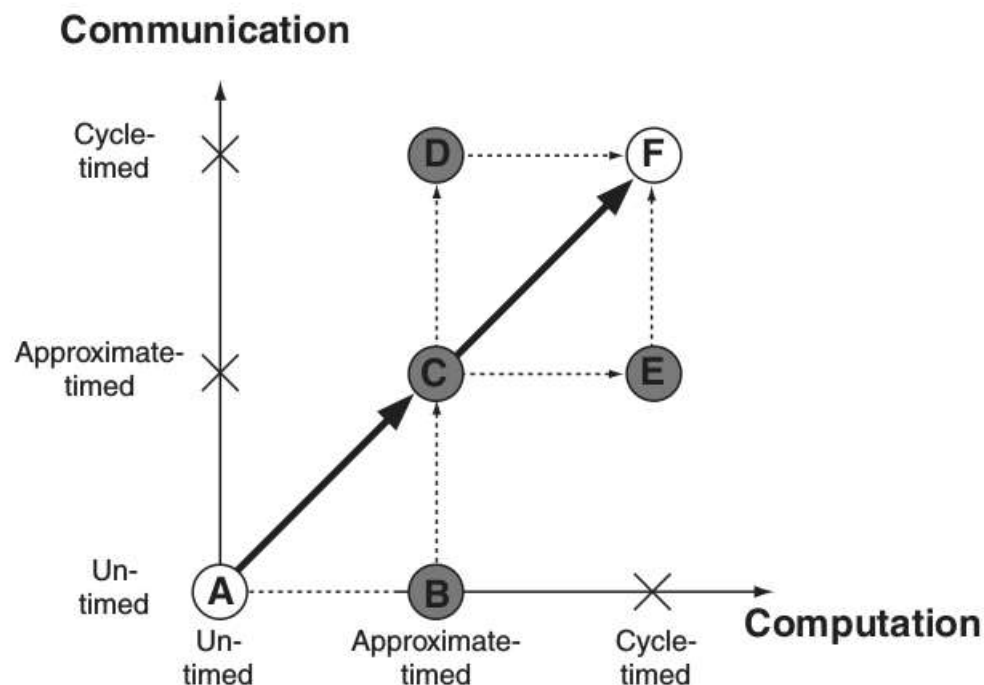
PS5 Physical View



- Partitioning a concurrent application onto architectural modules can be performed in two main ways:
 - Successive refinement
 - Explicit mapping

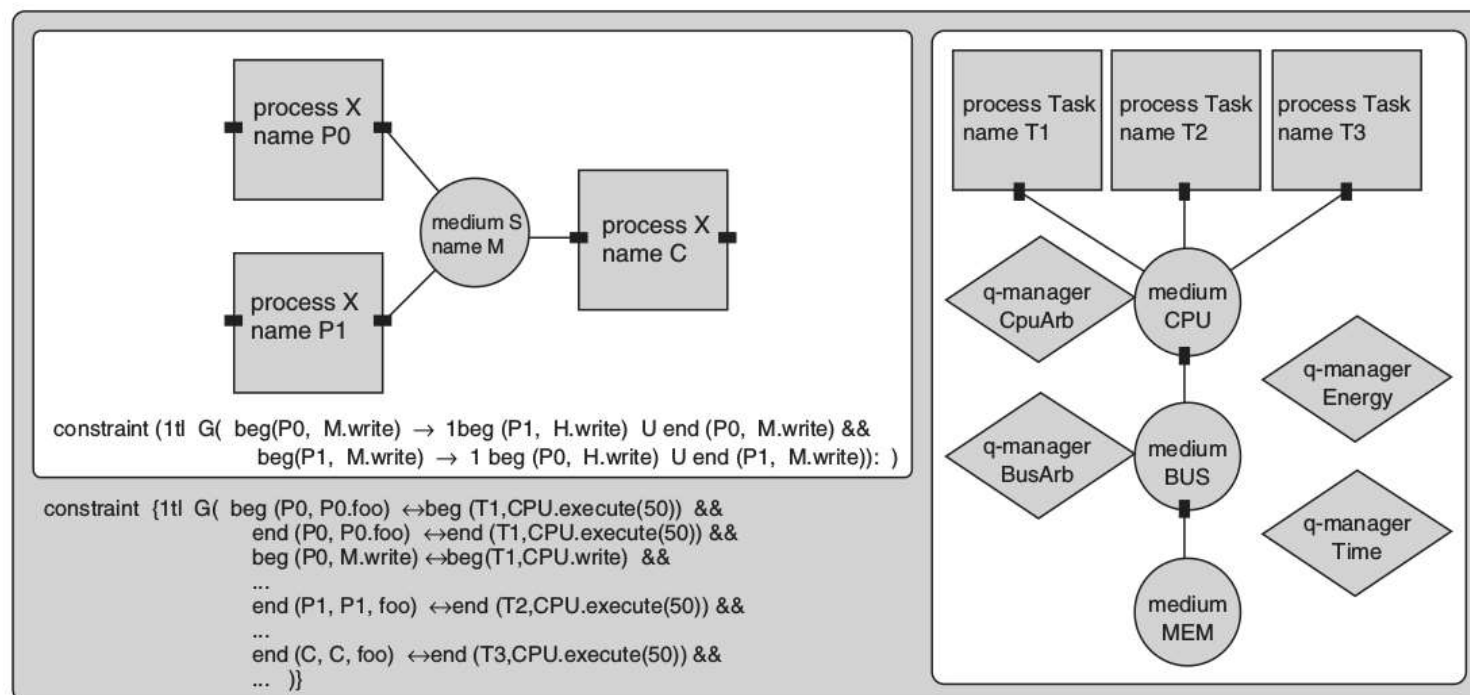
Refinement-Based Methods

- Additional information for communication and computation

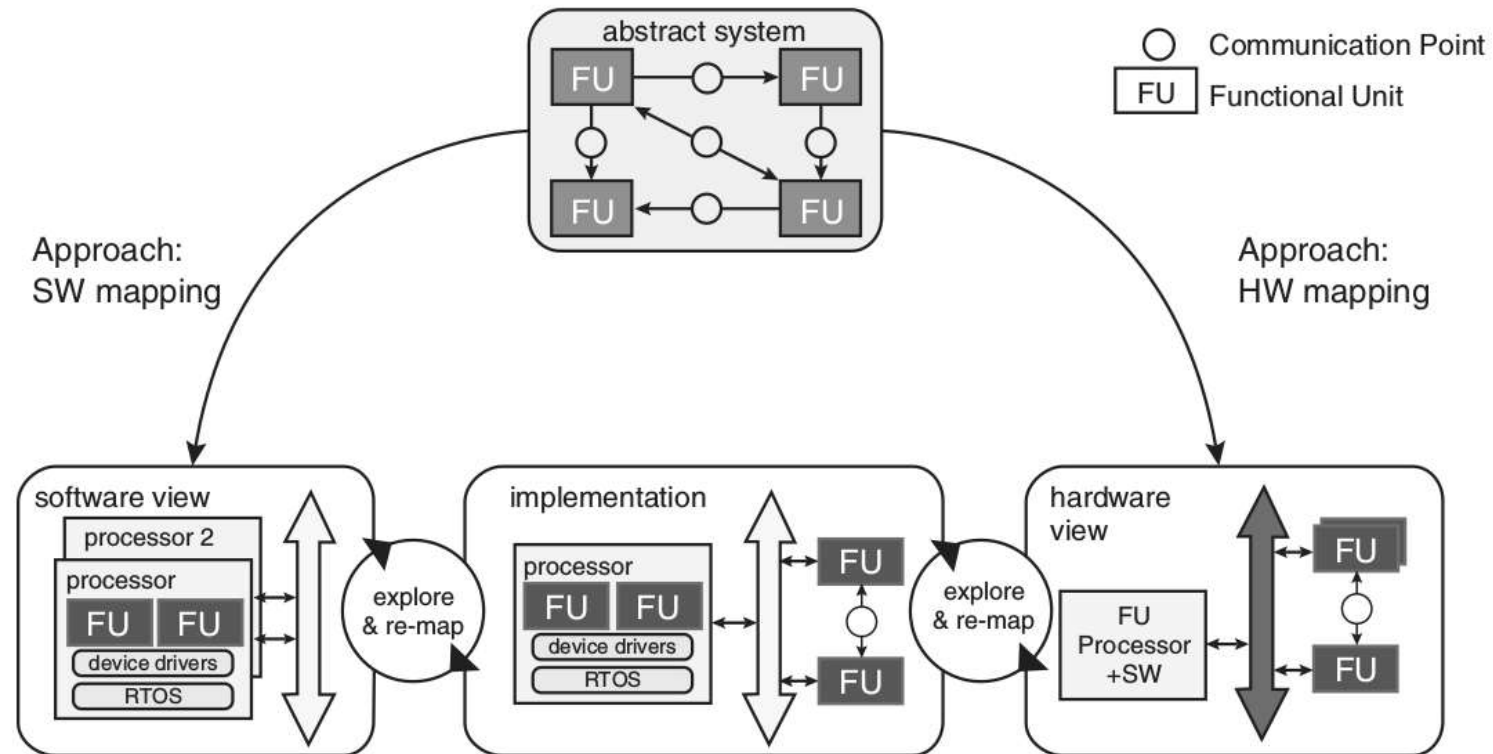


- A - untimed functional model
- B - adding architectural information - component assembly
- C - bus arbitration model
- D - behavioral bus-functional model
- E - the cycle-accurate computation model
- F - RTL implementation

Partitioning - Explicit Mapping-Based Methods



Functionality assignment process - Approaches



Hardware Partitioning

- Hardware design at the system mostly a platform configuration activity
- At the unit level, everything is still the same.
- Software verification becomes much more important.
- Formally checking the absence of bugs - requirement

Software Partition

- Partitioning is moving from a hardware-centric or hardware-biased approach to a software-centric approach
- Most of the functions are implemented on processors
- Timing-critical or power-demanding - hardware components
- Embedded processors are very powerful (as desktop)
- Processors might execute multiple tasks -> an OS is often needed
- These OSs are often very similar to their desktop counterparts
- The software architecture important as hardware architecture.
- A well-designed software architecture efficiently exploit the hardware platform

Software Partition - Partitioning over Multiple Processors

- Assigning portions of the overall software to the various processors
- Symmetric or heterogeneous processors
- Using more than one processor -> better functions concurrency
- Two tasks need to exchange a significant amount of data ->
- Application-specific communication mechanisms

Software Partition - Partitioning over Multiple Tasks

- Multiple tasks, processes, or threads
- A scheduling algorithm
- Context switching
- Communication overheads

Software Partition - Operating System

- Generally, most embedded systems **require some kind of OS** to manage a processor or several processors efficiently.
- An OS, virtualizes a set of processors to a set of tasks
- Arbitrates the access to shared hardware resources
- Enable memory allocation
- Task protection
- Implement communication
- Software partitioning take into account the OS characteristics

Memory Partitioning

- Distributing code and data across multiple memory subsystems
- The virtualization of large, slow, inexpensive with small, fast, relatively expensive
- Caching techniques

Communication Implementation

- Communication - the performance bottleneck
- Two implementation approaches:
 - Template instantiation.
 - Interface synthesis.

Communication Implementation - Template Instantiation

- Matches one or more templates against functional and performance requirements
- If a SIMD processor needs an interface to a general-purpose CPU
- A processor-to-processor template may be instantiated
 - pipelined vs serial
 - burst
 - 32-bit vs 64-bit

Communication Implementation - Interface Synthesis

- IPXACT generators
- Has several requirements before it can be applied
- The average throughput of the transmitting and receiving interfaces must be the same - no deep buffers are used
- The signal-level transmission protocol must have FSM semantics
 - Automata-based
 - Language-based synthesis
- A number of strategies are available to synthesize a communication interface